

SAHAYAK

RPwD Application Preparation — Shared Schema & API Blueprint

Purpose: This document is the shared technical contract for the five-person hackathon team. It defines the common data structures, API contracts, AI-agent behavior, PDF-generation flow, and S3 storage conventions for the RPwD application-preparation feature.

Feature scope: The user provides information conversationally by voice or text. Sahayak extracts the information, asks only for missing details, validates the structured application data, and generates a downloadable PDF summary/application-preparation package. The PwD applicant can take this PDF to a cyber center or authorized operator, reducing the need to verbally repeat the same information.

Important boundary: Sahayak prepares information. It does not claim to issue a disability certificate, medically assess the applicant, submit the government application, complete authentication, solve CAPTCHA, or handle OTPs/passwords.

Version 1.0 — Shared Team Contract

1. System Goal

The RPwD feature should move the applicant through the following workflow:

```
Voice / Text User Input
↓
Speech-to-Text (when voice is used)
↓
FastAPI / Chat Orchestrator
↓
Sahayak AI Agent
↓
Extract structured application fields
↓
Validate + identify missing information
↓
Ask only relevant follow-up questions
↓
Application data complete
↓
Generate PDF application summary/package
↓
Store generated PDF in S3
↓
Return PDF link / package metadata to Frontend
```

The core principle is that the LLM handles conversation and information extraction, while deterministic backend code owns validation, completion state, PDF generation, and storage.

2. Master Shared Data Schema

The canonical object for this feature is **RPwDApplication**. All team members should use these exact field names unless the team formally changes the shared contract.

```
RPwDApplication
■■■ application_id
■■■ session_id
■■■ status
■■■ applicant
■   ■■■ first_name
■   ■■■ middle_name
■   ■■■ last_name
■   ■■■ father_name
■   ■■■ mother_name
■   ■■■ date_of_birth
■   ■■■ age
■   ■■■ gender
■■■ addresses
■   ■■■ permanent_address
■   ■■■ communication_address
■   ■■■ same_as_permanent
■   ■■■ communication_address_since
■■■ education_and_occupation
■   ■■■ educational_status
■   ■■■ occupation
■■■ identification
■   ■■■ identification_mark_1
■   ■■■ identification_mark_2
■■■ disability
■   ■■■ disability_type
■   ■■■ disability_description
■   ■■■ onset_type
■   ■■■ onset_year
■■■ previous_application
■   ■■■ previously_applied
■   ■■■ authority
■   ■■■ district
■   ■■■ result
■■■ previous_certificate
■   ■■■ previously_issued
■   ■■■ certificate_available
■■■ documents
■   ■■■ residence_proof
■   ■■■ identity_document
■   ■■■ aadhaar_available
■   ■■■ passport_photos_available
```

- ■■■ medical_reports_available
- guardian
- ■■■ has_guardian
- ■■■ full_name
- ■■■ relationship_to_applicant
- ■■■ contact_number
- declaration
- ■■■ confirmed
- ■■■ application_place
- ■■■ generated_date
- metadata
 - created_at
 - updated_at
 - completion_percentage

3. Field-Level Contract

3.1 Applicant

Field	Type	Requirement	Notes
first_name	string	Required	Applicant's given name
middle_name	string null	Optional	Do not force if absent
last_name	string	Required when applicable	Family/surname
father_name	string	Required	Collect conversationally
mother_name	string	Required	Collect conversationally
date_of_birth	YYYY-MM-DD	Required	Canonical stored format
age	integer	Derived	Calculate from DOB; do not maintain conflicting values
gender	enum/string	Required	Map user answer to accepted form category

3.2 Address

Field	Type	Requirement	Notes
permanent_address	object	Required	house/address line, locality, village/town/city, district, state, PIN
communication_address	object null	Conditional	Required only when different from permanent address
same_as_permanent	boolean	Required	If true, copy/display permanent address
communication_address_sinc e	string/date	Conditional	Collect if relevant to communication address

3.3 Education, Occupation & Identification

Field	Type	Requirement	Notes
educational_status	enum	Required	Map natural language to supported category
occupation	string	Required	Student, employed, self-employed, unemployed, etc.
identification_mark_1	string null	Optional	Physical identification mark if required
identification_mark_2	string null	Optional	Second identification mark if applicable

3.4 Disability

Field	Type	Requirement	Notes
disability_type	string/enum	Required	Record applicant's stated disability; agent must not medically diagnose
disability_description	string null	Optional	Applicant's own description
onset_type	enum	Required	from_birth since_year
onset_year	integer null	Conditional	Required when onset_type = since_year

3.5 Previous Application / Certificate

Field	Type	Requirement	Notes
previously_applied	boolean	Required	Has the applicant applied before?
authority	string null	Conditional	Only if previously applied
district	string null	Conditional	Only if previously applied
result	string null	Conditional	Previous application result/status
previously_issued	boolean	Required	Has a disability certificate been issued before?
certificate_available	boolean null	Conditional	Only if previously issued

3.6 Documents

Field	Type	Requirement	Notes
residence_proof	object	Required	Document type + availability + optional S3 key
identity_document	object null	As needed	Track type/availability; avoid storing more than needed
aadhaar_available	boolean	Optional	Availability only for MVP unless official workflow requires more
passport_photos_available	boolean	Required	Tracks readiness; actual photo storage is optional
medical_reports_available	boolean	Optional	Track supporting records if user has them

3.7 Guardian & Declaration

Field	Type	Requirement	Notes
has_guardian	boolean	Required	Controls conditional guardian fields
full_name	string null	Conditional	Guardian/applicant representative
relationship_to_applicant	string null	Conditional	Relationship
contact_number	string null	Conditional	Validate format; avoid exposing unnecessarily
confirmed	boolean	Required before final PDF	Applicant confirms information
application_place	string	Required	Place shown in final package
generated_date	date-time	System generated	Set by backend/PDF generator

4. Exclusive AI Agent Design Rule — Do NOT Interrogate the User

One very important design decision: The agent should **not** ask all 30+ fields one by one in a robotic interrogation.

The agent must first extract every useful field from the user's natural language. Only after extraction and validation should it ask for information that is still missing.

```
User:

"I am Venu, 21 years old. I live in Bengaluru.

I have had visual impairment since birth."

Agent extracts automatically:

✓ Name
✓ Age
✓ City
✓ Disability
✓ Disability onset

Then only asks for missing fields:

"What is your permanent address?"

"What is your highest education level?"

"Have you applied for a disability certificate before?"
```

This is mandatory behavior for the AI-agent implementation. The agent must treat conversation as an information-extraction process, not as a static multi-page form.

Recommended agent loop:

1. Receive user message.
2. Extract any RPwDApplication fields present in the message.
3. Merge extracted values with the current application state.
4. Validate extracted values.
5. Determine required fields that are still missing.
6. Apply conditional rules:
 - If same_as_permanent = true, do not ask again for communication address.
 - If previously_applied = false, do not ask authority/district/result.
 - If onset_type = from_birth, do not ask onset_year.
 - If has_guardian = false, do not ask guardian details.
7. Ask the single most relevant next question.
8. Repeat until minimum application data is complete.
9. Show summary and request declaration confirmation.
10. Generate PDF only after confirmation.

5. Application State & Completion Rules

Status	Meaning
draft	Application session created; information may be incomplete
collecting_information	Agent is extracting/asking for missing data
ready_for_review	Required fields complete; show user summary
confirmed	User confirms declaration
pdf_generated	Application-preparation PDF successfully generated
error	Recoverable generation/validation error

Do not allow PDF generation when required data is incomplete. Optional and conditional fields should be clearly marked as unavailable/not applicable where appropriate.

6. Shared API Contract

Keep the browser-facing API small. The frontend should not directly call internal agent tools. The backend orchestrates the agent, validation, state, PDF generation, and S3 operations.

POST /chat

```
Request
{
  "session_id": "optional-uuid",
  "message": "I am a BSc student...",
  "language": "en"
}

Response
{
  "session_id": "uuid",
  "assistant_message": "Thank you. What is your permanent address?",
  "application": { "...partial RPwDApplication..." },
  "missing_fields": ["permanent_address", "educational_status"],
  "next_question": "What is your permanent address?",
  "status": "collecting_information"
}
```

GET /applications/{application_id}

```
Response
{
  "application_id": "uuid",
  "status": "collecting_information",
  "application": { "...RPwDApplication..." },
  "completion_percentage": 68,
  "missing_fields": ["occupation", "declaration.confirmed"]
}
```

PATCH /applications/{application_id}

```
Request
{
  "updates": {
    "education_and_occupation": {
      "educational_status": "Graduate"
    }
  }
}

Response
{
  "application": { "...updated RPwDApplication..." },
  "missing_fields": ["occupation"]
}
```

POST /applications/{application_id}/validate

```
Response
{
  "valid": false,
  "missing_fields": ["occupation"],
  "invalid_fields": [],
  "conditional_requirements": []
}
```

POST /applications/{application_id}/confirm

```
Request
{
  "confirmed": true,
  "application_place": "Bengaluru"
}

Response
{
  "status": "confirmed",
  "ready_for_pdf": true
}
```

POST /applications/{application_id}/generate-pdf

```
Response
{
  "application_id": "uuid",
  "status": "pdf_generated",
  "pdf_object_key": "applications/{id}/generated/rpwd_application.pdf",
  "download_url": "signed-or-temporary-url",
  "generated_at": "ISO-8601"
}
```

GET /applications/{application_id}/pdf

```
Response
{
  "status": "pdf_generated",
  "download_url": "signed-or-temporary-url",
  "expires_at": "ISO-8601"
}
```

7. Internal Tool Contracts

Tool	Input	Output	Owner
extract_application_fields	User message + current state	Partial field updates + confidence	AI/Backend
get_missing_fields	RPwDApplication	Missing required/conditional fields	Backend
validate_application	RPwDApplication	Validation result/errors	Backend
merge_application_updates	Current state + extracted updates	Updated canonical state	Backend
generate_application_pdf	Confirmed RPwDApplication	PDF bytes + metadata	Application/PDF Engine
store_application_package	PDF/document bytes	S3 object key + metadata	Backend/Storage

8. S3 Storage Design

S3 should be treated as **object storage**, not as a relational database. Store generated PDFs and optionally uploaded documents in S3. Store application state/structured metadata in the backend's chosen persistence layer; if the MVP stores JSON objects only, keep a clear canonical JSON object per application and avoid pretending S3 provides relational querying.

```
sahayak-rpwd/  
■  
■■■ applications/  
■   ■■■ {application_id}/  
■       ■■■ application.json  
■       ■■■ uploads/  
■           ■■■ residence-proof/  
■           ■■■ previous-certificate/  
■           ■■■ generated/  
■               ■■■ rpwd_application_summary.pdf  
■  
■■■ temp/  
■       ■■■ {session_id}/
```

Recommended object naming:

```
applications/{application_id}/application.json  
applications/{application_id}/uploads/{document_type}/{file_id}  
applications/{application_id}/generated/rpwd_application_summary.pdf
```

Security rule: never expose permanent public URLs for sensitive application data. Prefer short-lived signed URLs for download/upload where applicable.

9. Five-Person Integration Ownership

Person	Primary Work	Must Consume
1 — Orchestrator / Backend / Agent	Session, orchestration, state merge, API routing, integration	Master schema + all contracts
2 — Knowledge / Form Intelligence	Field guidance, explanations, source/form mapping	Field names + validation metadata
3 — Voice / Accessibility	STT/TTS, accessible interaction, language flow	POST /chat contract
4 — Application Engine	Validation rules + PDF generation + document readiness	RPwDApplication canonical object
5 — Frontend	Conversation UI, progress, review, PDF download	API response models + status values

10. Integration Rules — Non-Negotiable

- No teammate should independently rename shared fields such as annual field names, addresses, disability values, or application status.
- All partial updates must be merged into the canonical RPwDApplication object by deterministic backend logic.
- The frontend displays backend-owned state; it must not invent completion state independently.
- The agent may suggest/extract values but validation belongs to deterministic code.
- The PDF generator consumes the canonical confirmed object only.
- The agent must never claim that the government application was submitted unless the system actually receives a submission confirmation.
- Do not collect OTPs, passwords, UPI PINs, or CAPTCHA solutions.
- Avoid retaining sensitive identity information longer than required for the demo/workflow.
- Every branch should import or reference the same shared schema package/version.

11. Recommended Repository Structure

```
sahayak/
├── backend/
│   ├── main.py
│   ├── api/
│   │   ├── chat.py
│   │   ├── applications.py
│   │   ├── agent/
│   │   │   ├── orchestrator.py
│   │   │   ├── prompts.py
│   │   ├── services/
│   │   │   ├── application_state.py
│   │   │   ├── validation.py
│   │   │   ├── pdf_generator.py
│   │   │   ├── storage.py
│   │   ├── schemas/
│   │   │   ├── rpwd_application.py
│   │   │   ├── api_models.py
│   │   │   └── enums.py
│   └──
├── frontend/
├── knowledge/
├── rpwd/
├── shared/
│   ├── schema-contract/
│   └── README.md
├── tests/
│   ├── test_validation.py
│   ├── test_state_merge.py
│   └── test_api_contract.py
```

12. Final End-to-End Demo Flow

1. User speaks/types naturally.
2. Agent extracts multiple details from one message.
3. Backend merges extracted values into RPwDApplication.
4. Backend validates the current state.
5. Agent asks only the next missing relevant question.
6. Conditional questions are skipped when not applicable.
7. User reviews the collected summary.
8. User confirms the declaration.
9. Backend generates the RPwD application-preparation PDF.
10. PDF is stored in S3.
11. Frontend provides the downloadable package.
12. User can take the PDF to a cyber center/operator and avoid repeating the collected details verbally.

Definition of Done for This Feature

- User can start by voice or text.
- Agent extracts multiple fields from natural language.
- Agent asks only for missing required information.
- Conditional fields are handled correctly.
- A canonical RPwDApplication object is maintained.
- Application state can be reviewed before generation.
- PDF is generated only after confirmation.
- Generated PDF is stored safely and returned through the API.
- The demo never claims to issue or submit a disability certificate.